

# Context Engineering

Language Models

# Course Contents

---

Deep Learning

---

Transformer

---

LLMs

---

**Context Engineering**

---

Finetuning

---

Agents

---

The Bigger Picture

# This Part: Context Engineering

- in-context learning
- prompt engineering
- RAG

# In-Context Learning (ICL)

the model's ability to condition its outputs on patterns, instructions, or demonstrations present in the context window, *without updating parameters* (in contrast to fine-tuning)

→ not really learning (rather inference-time pattern recognition: locating and using relevant learnings from training)

ICL is the emergent phenomenon that makes prompting powerful.

# Few-Shot ICL

zero-shot ICL:

no examples, just instructions

→ instruction following: model infers task purely from natural-language description

few-shot ICL:

conditioning on a few input-output examples

→ demonstration-based: model “learns” from examples

The three settings we explore for in-context learning

## Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese =>                    ← prompt
```

## One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer   ← example
3 cheese =>                    ← prompt
```

## Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer   ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese =>                    ← prompt
```

Traditional fine-tuning (not used for GPT-3)

## Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



GPT-3

large context length also allows many-shot ICL

# Prompt Engineering

prompt: user interface to the model

prompt engineering: shape model behavior using well-structured prompts

difference from traditional user interfaces:

probabilistic, not deterministic (shape the *distribution* of the model's responses)

→ good prompts can improve a model's performance and reduce hallucination, bias, or inefficiency

[prompt engineering](#), [GPT guide](#)

# Typical Prompt Structure

prompt = instruction + context + query + examples + constraints + output cue

task definition: tells the model what to do

input/problem: content you want the model to work on

formatting/style/scope: specify limits (length, tone, format, ...)

indicator: signal that tells the model where to start responding

optional background information: provides grounding or domain-specific details (external knowledge, role assignment, scenario setup)

few-shot component: show the model what good outputs look like

# Example Prompt

Summarize the following text in one sentence.

Text: "Large language models show emergent abilities when scaled."

Answer:

- instruction: *Summarize the following text in one sentence.*
- query (input/problem): *Large language models show emergent behavior when scaled.*
- constraint: implicit → *in one sentence*
- output cue: *Answer:*



# Simplest Possible Prompt

often query phrased as instruction:

Summarize: Large language models show emergent abilities when scaled.

but without instruction model just completes/continues the query text (usually not really useful):

Large language models show emergent abilities when scaled.

At the bare minimum, you only need the query. But for reliable, controllable outputs, you usually need to scaffold it with instruction, context, constraints, etc.

# Best Practices: Clear Instruction & Query

Tell me about photosynthesis.

better:

Explain photosynthesis in simple terms suitable for a 12-year-old, using a short paragraph and one analogy.

→ clearly state what you want the model to do (best when specific, unambiguous, and outcome-focused)

# Best Practices: Give Context

Write about the causes of the French Revolution.

better:

You are a historian writing a study guide for undergraduate students.  
Write a concise explanation of the causes of the French Revolution, highlighting 3 major factors.

context: *You are a historian writing a study guide for undergraduate students.*

→ role assignment and audience specification steer tone and depth

# Aka Role Prompting

A Role Prompt

You are  
Shakespeare, an  
English writer.  
Write me a poem.

Model Output

Of lovers' hearts  
and passion's  
fire...

# Best Practices: Specify Constraints

List some facts about Mars.

better (if you want specific outputs):

List exactly 3 scientifically verified facts about Mars.

Output them in JSON format with keys: "fact1", "fact2", "fact3".

# Output Indicators

earlier models (pre-ChatGPT) were very sensitive to formatting (training datasets contained clear separators)

→ certain cues like *Answer:*, *A:*, *Response:*, or even *###* dramatically affected performance (better alignment with training data)

not strictly required anymore for modern instruction-tuned LLMs

but can still be useful for automation or complex prompts (improve consistency, reduce ambiguity):

- classification:

Label:

- code generation:

Code:

# Examples/Demonstrations

Convert these temperatures from Celsius to Fahrenheit:

25, 30, 35

few-shot examples help for unusual, complex, or under-specified tasks:

Convert these temperatures from Celsius to Fahrenheit:

C: 0  $\rightarrow$  F: 32

C: 10  $\rightarrow$  F: 50

C: 20  $\rightarrow$  F: 68

Now continue for:

C: 25, 30, 35

# Important Subtlety: Formatting & Abstraction

minimum effect of examples: enforce formatting patterns (small models)

C: 25 → F: 666

maximum: approximate transformation rule behind examples (large models)

C: 25 → F: 77

But not even the largest LLMs actually derive  $F = C \cdot 1.8 + 32$ .

instead: interpolation from similar training cases memorized in model weights  
(not directly interpolating the numerical values, but its vector representations)

→ looks like reasoning, but is just probabilistic pattern completion

→ no algorithmic guarantee (typically breaks for unusual inputs)



# Structured Design

So, prompts aren't random — they have designable components.

different tasks emphasize different sections:

- creative writing → role/context
- (approximate) reasoning → examples
- structured output → constraints

# Advanced Prompting Paradigms

- chain-of-thought prompting (explicit reasoning steps)
- self-consistency (aggregation of multiple sampled reasoning paths) and sampling strategies (determine how diverse those paths are)
- role-playing and system prompts (e.g., teacher, critic, planner)
- scaffolding: multi-turn prompt orchestration, planning, subtask decomposition
- meta-prompts / prompt templates (using one prompt to generate another)
- prompt chaining (e.g., step 1: extract facts → step 2: summarize → step 3: critique)
- automated prompt generation and optimization

# Hallucinations

knowledge encoded implicitly in weights rather than as explicit facts  
→ probabilistic pattern completion can produce hallucinated facts

**Instruction:** What are Thomas Edison's main contributions to science and technology?

**Response:** Thomas Edison developed **the first practical telephone** and **invented** the light bulb.

The response is factually incorrect. In reality, **Edison improved the light bulb, building on earlier designs by others, and Alexander Graham Bell invented the telephone.**

can be mitigated by retrieving the corresponding information from an external knowledge base and pasting it in context  
(prompt engineering → context engineering)

# Retrieval Augmented Generation (RAG)

- vector similarity
- keyword
- hybrid



